DATA 620 Week 4 Assignment

Centrality Measurement

Based on the U.S. Congressional Bill Cosponsorship Network

Data Source: Congress.gov API (Library of Congress)
Implementation: Python • NetworkX • Requests

Candace Grant
CUNY School of Professional Studies – M.S. Data Science
March 2026

1. Project Description

This document identifies an interesting network dataset available through a web API, describes a plan for loading and preparing the data for network analysis, and presents hypothetical outcomes that could be predicted from comparing degree centrality across categorical groups. The dataset selected is the U.S. Congressional bill cosponsorship network, derived from the Congress.gov API maintained by the Library of Congress.

In this network, legislators are connected when they cosponsor the same piece of legislation. Cosponsorship is one of the most revealing forms of political collaboration: when a legislator adds their name to a bill sponsored by a colleague, it signals political alignment, coalition-building, or strategic positioning. Analyzing the resulting network with centrality measures—and comparing those measures across categorical groups such as party affiliation—can reveal structural patterns about who collaborates most broadly, who bridges partisan divides, and whether party membership predicts a legislator's connectivity within the broader legislative network.

2. Identified Data Source

2.1 The Congress.gov API (Library of Congress)

The data source for this project is the Congress.gov API, a free and publicly accessible RESTful API maintained by the Library of Congress. The API is available at https://api.congress.gov and provides structured JSON responses containing comprehensive metadata on bills, amendments, members, votes, committees, nominations, and treaties across the full history of the United States Congress.

The API requires a free API key, obtainable by registering at https://api.data.gov. Authentication is straightforward: the API key is passed as a query parameter (api_key) with every request. The rate limit is 5,000 requests per hour, which is generous enough to collect all necessary data for a single Congressional session in one sitting. The API supports pagination via limit (up to 250 results per call) and offset parameters.

2.2 Why This Dataset Is Suitable

This dataset satisfies all requirements for the assignment for several reasons. First, the data is acquired entirely through a web API, meeting the web scraping or web API requirement. Second, each node (legislator) has multiple categorical variables available, including party affiliation (Democrat, Republican, Independent), chamber (House or Senate), state, and region. Third, the cosponsorship relationships create a naturally occurring, well-defined network with meaningful edges. Fourth, this is a well-studied network in political science, meaning the results can be grounded in established research literature. Finally, the data is rich enough to support

degree centrality comparison across groups while being manageable in size (approximately 535 legislators per Congressional session).

2.3 Key API Endpoints

The following three API endpoints will be used to acquire the data needed for constructing the cosponsorship network:

API Endpoint	Purpose
GET /v3/member	Retrieve the list of all members of Congress for a given session. Returns each member's name, bioguide ID, party, state, chamber, and district. This provides the node list and all categorical attributes.
GET /v3/bill/{congress}/{type}	Retrieve all bills introduced in a given Congress (e.g., the 118th Congress, 2023–2024). Returns bill ID, title, sponsor, number of cosponsors, subjects, and status. This is the starting point for identifying bills with cosponsors.
GET /v3/bill/{congress}/{type}/{number}/cosponsors	For a specific bill, retrieve the full list of cosponsors with their bioguide IDs and party information. This endpoint provides the co-occurrence data needed to build edges: the bill sponsor and all cosponsors form a collaboration group.

3. Network Definition

The network is defined as follows:

Element	Definition
Nodes	Members of the U.S. Congress (Senators and Representatives). For the 118th Congress, this is approximately 535 legislators.
Edges	An undirected edge connects two legislators if they cosponsored at least one bill together. Specifically, an edge is formed between the sponsor of a bill and each of its cosponsors, as well as between all pairs of cosponsors on the same bill.
Edge Weight	The number of distinct bills that a pair of legislators cosponsored together. A higher weight indicates a stronger and more sustained collaboration relationship.
Degree Centrality	The fraction of other legislators in the network that a given legislator is directly connected to via cosponsorship. Computed as DC(v) = degree(v) / (n – 1). Higher degree centrality indicates broader legislative collaboration reach.

4. Categorical Variables

Each legislator node carries several categorical attributes obtained directly from the Congress.gov API member endpoint. The primary categorical variable for degree centrality comparison is party affiliation, with additional variables available for secondary analysis:

Variable	Categories	Analytical Relevance
Party Affiliation (Primary)	Democrat, Republican, Independent	The most politically significant grouping. Enables testing whether one party's legislators collaborate more broadly than the other, and whether Independents serve as cross-party bridges.
Chamber	Senate, House	Structural comparison between the 100-member Senate and 435-member House. Senators may have higher normalized centrality due to smaller chamber size and more cross-committee interaction.
Geographic Region	Northeast, South, Midwest, West	Tests whether legislators from certain regions are more collaborative, potentially reflecting regional caucus dynamics or state delegation behaviors.
State	50 states + territories	Finer-grained geographic grouping. Large state delegations (CA, TX, NY) may exhibit higher within-group density due to shared state interests.

5. High-Level Plan for Loading Data

The following step-by-step plan describes how the data would be loaded from the Congress.gov API and transformed into a NetworkX graph object ready for degree centrality analysis. This plan is designed for implementation in Python.

Step 1 – Environment Setup

Install and import the required Python libraries: requests for making authenticated HTTP calls to the Congress.gov API, json for parsing responses, pandas for organizing the data into tabular DataFrames, networkx for constructing the graph and computing centrality measures, itertools for generating pairwise combinations from cosponsor lists, and matplotlib with seaborn for visualization. Store the API key in an environment variable or a separate config file, never hardcoded in the analysis script.

```
import requests, json, pandas as pd, networkx as nx
from itertools import combinations
import matplotlib.pyplot as plt, seaborn as sns
API_KEY = os.environ['CONGRESS_API_KEY']
BASE_URL = 'https://api.congress.gov/v3'
```

Step 2 – Retrieve the Member List (Node Data)

Call the /v3/member endpoint with the congress parameter set to the target session (e.g., 118 for the 118th Congress). Paginate through results using limit=250 and incrementing the offset until all members are collected. For each member, extract: bioguide_id (unique identifier), name, party (Democrat, Republican, Independent), state, district (for House members), and chamber (Senate or House). Derive a region variable by mapping each state to its Census region (Northeast, South, Midwest, West). Store all member data in a pandas DataFrame called members_df. This DataFrame represents the node list with all categorical attributes.

Step 3 – Retrieve Bills and Their Cosponsors (Edge Data)

Call the /v3/bill/{congress}/{type} endpoint for both bill types ‘hr’ (House) and ‘s’ (Senate) to retrieve all bills introduced in the target Congress. For each bill, record the bill ID, sponsor bioguide ID, and cosponsor count. Then, for each bill that has at least two cosponsors, call the /v3/bill/{congress}/{type}/{number}/cosponsors endpoint to retrieve the full list of cosponsor bioguide IDs. Store the results in a DataFrame called bills_df with columns: bill_id, sponsor_id, and cosponsor_ids (a list).

To manage API rate limits, implement a helper function with a time.sleep(0.5) delay between calls and a check on the X-RateLimit-Remaining header. Cache all raw API responses as local JSON files so the collection only needs to run once. For a typical Congressional session with 10,000–15,000 bills, the cosponsors endpoint calls will require the most requests; focus on bills with at least 2 cosponsors to ensure every bill contributes at least one edge.

Step 4 – Construct the Edge List

For each bill, assemble the full collaboration group: the sponsor plus all cosponsors. Using Python's itertools.combinations(collaboration_group, 2), generate all pairwise combinations of legislators involved with that bill. Each pair becomes an edge. If the same pair appears across multiple bills, the edge weight is incremented. Aggregate these pairs into an edge list DataFrame called edges_df with columns: legislator_1, legislator_2, and weight (the count of shared bills).

```
# Pseudocode for edge construction
for bill in bills_df.itertuples():
    group = [bill.sponsor_id] + bill.cosponsor_ids
    for pair in combinations(group, 2):
        edge_counts[pair] += 1
```

Step 5 – Build the NetworkX Graph

Convert edges_df into an undirected weighted NetworkX graph using nx.from_pandas_edgelist(edges_df, source='legislator_1', target='legislator_2', edge_attr='weight'). Attach categorical node attributes from members_df using nx.set_node_attributes(). Verify the graph structure by printing summary statistics: node count, edge count, density, and number of connected components. The expected result is a single large connected component containing nearly all 535 legislators.

```
G = nx.from_pandas_edgelist(edges_df, 'legislator_1', 'legislator_2',
edge_attr='weight')
nx.set_node_attributes(G, members_dict)  # party, state, chamber, region
```

Step 6 – Data Validation and Cleaning

Perform quality checks: remove any self-loops, verify all nodes have categorical attributes assigned (handle any missing values by cross-referencing the member endpoint), confirm the distribution of nodes across party groups is reasonable (expect roughly 215–225 Democrats, 215–225 Republicans, and 2–5 Independents in the House and Senate combined), and optionally extract the largest connected component to ensure centrality measures are computed on a connected subgraph.

Step 7 – Compute Degree Centrality and Aggregate by Group

Compute degree centrality for every node using nx.degree_centrality(G). Merge the centrality values into a DataFrame with the categorical attributes. Use pandas groupby('party') to compute summary statistics per group: mean, median, standard deviation, min, max, and count. This summary table reveals the first descriptive differences across parties.

```
centrality = nx.degree_centrality(G)
centrality_df = pd.DataFrame.from_dict(centrality, orient='index',
columns=['degree_centrality'])
```

```
centrality_df = centrality_df.merge(members_df, left_index=True,
right_on='bioguide_id')
```

Step 8 – Cache and Serialize

Save all processed data locally: members_df and edges_df as CSV files, the NetworkX graph as a GraphML file using nx.write_graphml(G, 'congress_cosponsorship.graphml'), and the centrality_df as a CSV. This ensures full reproducibility without re-querying the API and provides the data files needed for the GitHub repository submission.

6. Hypothetical Outcomes: Predictions from Comparing Degree Centrality Across Party Affiliation

The following hypothetical outcomes describe plausible results that could emerge from comparing degree centrality across the primary categorical variable (party affiliation: Democrat, Republican, Independent). Each outcome includes the predicted statistical finding, a substantive interpretation grounded in political science, and suggested follow-up analyses. These outcomes demonstrate the types of insights that degree centrality comparison can yield in a political network.

6.1 Outcome A – The Minority Party Exhibits Higher Mean Degree Centrality

Prediction: A Kruskal-Wallis test comparing degree centrality distributions across Democrat, Republican, and Independent groups yields a statistically significant result ($p < 0.05$). Post-hoc pairwise comparisons reveal that the minority party (whichever party holds fewer seats in that chamber) has a significantly higher mean degree centrality than the majority party.

Interpretation: This outcome would suggest that minority party legislators cosponsor more broadly across the aisle as a strategic necessity. When a party lacks the votes to pass legislation on its own, its members must build coalitions by cosponsoring bills with members of the majority party. This forces minority party legislators to connect with a wider range of colleagues, inflating their degree centrality. Majority party members, by contrast, can often pass legislation with intra-party support alone, reducing their incentive to build cross-party ties. This finding would align with political science theories of coalition-building under divided or competitive legislative conditions.

6.2 Outcome B – No Significant Difference Between Parties, but Independents Are Extreme Outliers

Prediction: The Kruskal-Wallis test comparing Democrats and Republicans shows no significant difference ($p > 0.10$). However, the small group of Independent legislators (typically 2–4 members in modern Congresses, such as Senators Bernie Sanders and Angus King) show dramatically higher degree centrality than both parties' medians—potentially in the top 5% of all legislators.

Interpretation: Independents occupy a unique structural position in the cosponsorship network. Because they are not bound by party discipline, they are free to cosponsor legislation from both sides of the aisle. More importantly, Independents who caucus with one party (as Sanders and King caucus with Democrats) serve as visible bipartisan signals—a Republican legislator may be more willing to cosponsor a bill that already has an Independent's name on it. Their high

degree centrality would reflect their role as structural bridges between the two major party clusters in the network. Even though they are a tiny group (too small for robust statistical testing), their individual centrality scores are analytically significant.

6.3 Outcome C – Degree Centrality Varies More by Chamber Than by Party

Prediction: When degree centrality is compared across party affiliation, the effect is modest (small effect size, $\eta^2 < 0.03$). However, when the same comparison is run across chamber (Senate vs. House), a much larger and more significant difference emerges ($\eta^2 > 0.10$), with Senators showing substantially higher normalized degree centrality than House members.

Interpretation: This outcome would indicate that the structural properties of the chamber matter more than party identity for predicting collaboration breadth. The Senate, with only 100 members, operates under rules that encourage broader coalition-building (the filibuster requires 60 votes to advance most legislation, forcing cross-party cooperation). House members, operating in a 435-person chamber with strict majority-rule procedures, have less incentive and fewer opportunities to cosponsor outside their immediate circle. The smaller Senate also means each Senator's potential connections represent a larger fraction of the chamber, naturally producing higher normalized centrality scores.

6.4 Outcome D – Bimodal Degree Distribution Within the Majority Party Reveals a Leadership-Backbencher Divide

Prediction: The majority party's degree centrality distribution is distinctly bimodal: a large cluster of low-centrality "backbencher" members who cosponsor primarily within their own party caucus, and a small but prominent cluster of high-centrality leaders (committee chairs, party leadership, senior members) who cosponsor with an unusually broad range of colleagues. The minority party's distribution is more uniform and unimodal.

Interpretation: In the majority party, institutional power creates a two-tier structure. Committee chairs and party leaders (Speaker, Majority Leader, Whips) are involved in crafting and shepherding legislation across multiple policy areas, requiring them to build broad coalitions and accumulate many cosponsorship ties. Rank-and-file majority party members, however, can afford to be more insular—they follow party leadership and primarily cosponsor within their ideological wing. In the minority party, the lack of institutional power flattens this hierarchy: every member must actively seek cosponsorship opportunities, producing a more uniform centrality distribution. This finding would reveal how institutional roles, not just party identity, shape network position.

6.5 Outcome E – Legislators from Competitive (Swing) States Have Higher Centrality Than Those from Safe States

Prediction: When the geographic region variable is refined into a “state competitiveness” categorical variable (Swing State vs. Safe State, defined by recent presidential election margins), legislators from swing states show significantly higher degree centrality than those from safe states, regardless of party affiliation.

Interpretation: Legislators from competitive states face electoral pressure to demonstrate bipartisan cooperation to their constituents. A senator from a swing state like Pennsylvania, Arizona, or Georgia cannot afford to appear purely partisan; broad cosponsorship activity provides evidence of working across the aisle that can be cited in re-election campaigns. Legislators from safe states (e.g., deep-red or deep-blue districts) face no such pressure and may even be incentivized to avoid bipartisan cosponsorship to prevent primary challenges from their ideological flank. This outcome would demonstrate that electoral incentives, mediated through state-level competitiveness, shape network behavior in measurable ways.

7. Tools and Technology Stack

Category	Tool / Package	Role
Language	Python 3.10+	Primary language for API calls, data processing, graph analysis, and visualization.
API Interaction	requests	HTTP GET requests to Congress.gov API with API key authentication, pagination, and rate limiting.
Graph Analysis	NetworkX 3.x	Graph construction (from_pandas_edgelist), degree centrality (degree_centrality), and network statistics.
Data Wrangling	pandas	DataFrame operations for member data, edge lists, group aggregation via groupby, and merging.
Statistical Testing	scipy.stats, scikit-posthocs	Kruskal-Wallis (kruskal), Mann-Whitney U (mannwhitneyu), Dunn's post-hoc (posthoc_dunn).
Visualization	matplotlib, seaborn	Box plots, violin plots, network graph layouts, and heatmaps for statistical comparison.
Version Control	Git / GitHub	Repository for final submission including this plan document, code notebooks, and data files.

8. Summary

This plan identifies the U.S. Congressional bill cosponsorship network as the target dataset, acquired through the Congress.gov API (Library of Congress). The network consists of approximately 535 legislator nodes connected by cosponsorship edges, with party affiliation serving as the primary categorical variable for degree centrality comparison. The data loading plan describes eight concrete steps from API authentication through graph construction and centrality computation, all implemented in Python using the requests library for API interaction and NetworkX for graph analysis.

Five hypothetical outcomes were presented, each illustrating a distinct pattern that could emerge from comparing degree centrality across party groups: minority party strategic coalition-building, Independent legislators as structural bridges, chamber-level effects dominating party effects, leadership-backbencher divides within the majority party, and electoral competitiveness driving collaboration breadth. Each outcome is grounded in established political science theory and includes specific follow-up analyses that would deepen the investigation. Together, these outcomes demonstrate the analytical power of applying centrality measures to political networks and the range of substantive insights that can be extracted from comparing structural positions across categorical groups.

The complete analysis will be implemented and submitted as a GitHub repository containing this plan document, a Jupyter Notebook with executable Python code, cached data files, and a final report with visualizations and statistical results and a video presentation.